```c
#include <stdint.h>
#define _GNU_SOURCE

// Librearias estándar
#include <stdio.h>
#include <stdlib.h>
#include <stddef.h>
#include <string.h>

// Librerias criptográficas
#include <openssl/crypto.h>
#include <openssl/evp.h>
#include <openssl/core_names.h>

// Definiciones
#define LAB07_OK            0   // Valor de éxito
#define LAB07_ERR           1   // Error génerico
#define LAB07_ALG_ERR       2   // Error al asignar implementación de  algoritmo
#define LAB07_CTX_ERR       3   // Error al asignar contexto
#define LAB07_INT_ERR       4   // Error en mac init
#define LAB07_UPD_ERR       5   // Error en mac update
#define LAB07_FIN_ERR       6   // Error en mac final

// Prototipos de función
static int lab_compute_hmac(const unsigned char* key, size_t key_len, const
unsigned char* data, size_t data_len, unsigned char* out, size_t *out_len);
static int lab_compute_cmac(const unsigned char* key, size_t key_len, const
unsigned char* data, size_t data_len, unsigned char* out, size_t *out_len);
static int lab_compute_kmac(const unsigned char* key, size_t key_len, const
unsigned char* data, size_t data_len, const char *custom_str, unsigned char* out,
size_t out_len);

// Main
int main() {
    return LAB07_OK;
}

// Funciones


static int lab_compute_hmac(
    const unsigned char* key,
    size_t key_len,
    const unsigned char* data,
    size_t data_len,
    unsigned char* out,
    size_t *out_len
) {
```

```
    EVP_MAC *mac = NULL;
    EVP_MAC_CTX *ctx = NULL;
    OSSL_PARAM params[2];

    // Asignación de implmentación de MAC
    mac = EVP_MAC_fetch(NULL, "HMAC", NULL);
    if (!mac) {
        EVP_MAC_CTX_free(ctx);
        EVP_MAC_free(mac);
        return LAB07_ALG_ERR;    // Error al adjuntar el algoritmo HMAC
    }

    // Crear el contexto
    ctx = EVP_MAC_CTX_new(mac);
    if (!ctx) {
        EVP_MAC_CTX_free(ctx);
        EVP_MAC_free(mac);
        return LAB07_CTX_ERR;  // Error al asignar el contexto
    }

    params[0] = OSSL_PARAM_construct_utf8_string(OSSL_MAC_PARAM_DIGEST,
(char*)"SHA256", 0);
    params[1] = OSSL_PARAM_construct_end();

    if ( !EVP_MAC_init(ctx, key, key_len, params) ) {
        EVP_MAC_CTX_free(ctx);
        EVP_MAC_free(mac);
        return LAB07_INT_ERR;
    }
    if ( !EVP_MAC_update(ctx, data, data_len) ) {
        EVP_MAC_CTX_free(ctx);
        EVP_MAC_free(mac);
        return LAB07_UPD_ERR;
    }
    if ( !EVP_MAC_final(ctx, out, out_len, *out_len) )  {
        EVP_MAC_CTX_free(ctx);
        EVP_MAC_free(mac);
        return LAB07_FIN_ERR;
    }

    EVP_MAC_CTX_free(ctx);
    EVP_MAC_free(mac);
    return LAB07_OK;
};


static int lab_compute_cmac(
    const unsigned char* key,
```

```
    size_t key_len,
    const unsigned char* data,
    size_t data_len,
    unsigned char* out,
    size_t *out_len
){
    EVP_MAC *mac = NULL;
    EVP_MAC_CTX *ctx = NULL;
    OSSL_PARAM params[2];

    // Asignación de implmentación de MAC
    mac = EVP_MAC_fetch(NULL, "CMAC", NULL);
    if (!mac) {
        EVP_MAC_CTX_free(ctx);
        EVP_MAC_free(mac);
        return LAB07_ALG_ERR;    // Error al adjuntar el algoritmo HMAC
    }

    // Crear el contexto
    ctx = EVP_MAC_CTX_new(mac);
    if (!ctx) {
        EVP_MAC_CTX_free(ctx);
        EVP_MAC_free(mac);
        return LAB07_CTX_ERR;  // Error al asignar el contexto
    }

    params[0] = OSSL_PARAM_construct_utf8_string(OSSL_MAC_PARAM_CIPHER,
(char*)"AES-128-CBC", 0);
    params[1] = OSSL_PARAM_construct_end();

    if ( !EVP_MAC_init(ctx, key, key_len, params) ) {
        EVP_MAC_CTX_free(ctx);
        EVP_MAC_free(mac);
        return LAB07_INT_ERR;
    }
    if ( !EVP_MAC_update(ctx, data, data_len) ) {
        EVP_MAC_CTX_free(ctx);
        EVP_MAC_free(mac);
        return LAB07_UPD_ERR;
    }
    if ( !EVP_MAC_final(ctx, out, out_len, *out_len) )  {
        EVP_MAC_CTX_free(ctx);
        EVP_MAC_free(mac);
        return LAB07_FIN_ERR;
    }

    EVP_MAC_CTX_free(ctx);
    EVP_MAC_free(mac);
```

```
    return LAB07_OK;
};


static int lab_compute_kmac(
    const unsigned char* key, size_t key_len,
    const unsigned char* data, size_t data_len,
    const char *custom_str,
    unsigned char* out, size_t out_len
){
    EVP_MAC *mac = NULL;
    EVP_MAC_CTX *ctx = NULL;
    OSSL_PARAM params[3];
    int idx = 0;

    // Asignación de implmentación de MAC
    mac = EVP_MAC_fetch(NULL, "KMAC-128", NULL);
    if (!mac) {
        EVP_MAC_CTX_free(ctx);
        EVP_MAC_free(mac);
        return LAB07_ALG_ERR;    // Error al adjuntar el algoritmo HMAC
    }

    // Crear el contexto
    ctx = EVP_MAC_CTX_new(mac);
    if (!ctx) {
        EVP_MAC_CTX_free(ctx);
        EVP_MAC_free(mac);
        return LAB07_CTX_ERR;  // Error al asignar el contexto
    }

    if (custom_str) {
        params[idx++] = OSSL_PARAM_construct_utf8_string(OSSL_MAC_PARAM_CUSTOM,
(char*)custom_str, 0);
    }

    params[idx++] = OSSL_PARAM_construct_size_t(OSSL_MAC_PARAM_SIZE, &out_len);
    params[idx] = OSSL_PARAM_construct_end();

    if ( !EVP_MAC_init(ctx, key, key_len, params) ) {
        EVP_MAC_CTX_free(ctx);
        EVP_MAC_free(mac);
        return LAB07_INT_ERR;
    }
    if ( !EVP_MAC_update(ctx, data, data_len) ) {
        EVP_MAC_CTX_free(ctx);
        EVP_MAC_free(mac);
        return LAB07_UPD_ERR;
```

```
    }
    if ( !EVP_MAC_final(ctx, out, &out_len, out_len) )  {
        EVP_MAC_CTX_free(ctx);
        EVP_MAC_free(mac);
        return LAB07_FIN_ERR;
    }

    EVP_MAC_CTX_free(ctx);
    EVP_MAC_free(mac);
    return LAB07_OK;
};
```